Mathematics of AI and Data Science
Workshop Task

Task 1 - From the Notebook:

Dataset:

Students may choose one dataset:

- MNIST
- Fashion-MNIST
- CIFAR-10

Part 1 - Image Statistics(20%)

Compute the following statistics:

- Mean Image
- Mean Intensity
- Variance
- Standard Deviation
- Pixel Histogram

Questions:

- Which images are brighter?
- Which images have higher contrast?
- What does the histogram reveal?

Part 2 - Noise Analysis(15%)

Add Gaussian noise with different values of σ

Examples:

- $\sigma = 10$
- $\sigma = 25$
- $\sigma = 50$

Analyze:

- Visual changes
- Mean
- Variance
- Histogram

Part 3 - Convolution Mathematics(20%)

Implement convolution manually.
Do NOT use OpenCV or SciPy convolution functions.
Apply:

- Blur kernel
- Sharpen kernel
- Edge detection kernel

Explain mathematically:

- Why does each kernel produce its effect?
- What role do kernel values play?

Part 4 - Optimization Visualization(15%)

Dataset: Used Cars (Find on Kaggle).
Task: Price Prediction.
Create a simple optimization example.
For example:

- Fit [y = ax + b] using Gradient Descent.
- Plot Loss, Parameters, and Convergence.

The purpose is to visualize optimization rather than build a complex model.

Task 2 - Mentioned in the Workshop:

Implement the ‘im2col’ method yourself(without using any libraries) for the case of a 12×12 pixel image, and a 5×5 convolution kernel.

Mentor's Notes for the Tasks:

- Do not use special functions and/or libraries for implementing the statistical and noise analysis, optimization problem, or convolutional operations.
- For the statistical analysis part, you should calculate the variance, mean and other statistics manually! (Don't use numpy.mean).
- You CAN use numpy and/or SciPy for vector/matrix operations. Just don't use any high-level functions like scipy.signal.convolve2d, numpy.var, etc.
- Basically if you need a function, you should implement your own version of it, using numpy or scipy to make it more efficient (vectorized) than using mere 'Python for loops'.
- The same goes for the optimization problem: Define your own loss function, implement the gradient descent algorithm and … yourself, without using libraries like scikit-learn.
- Also the same story as for the 'im2col' task! (Who needs scipy?! :D)
- You need Gaussian noise? Write your own function! (Don't worry it's just 2 lines of code).
- You definitely CAN use plotting libraries such as matplotlib or seaborn.
- **IMPORTANT:** For visualization purposes, select a handful of images from the chosen dataset and visualize what's asked of you. It's better to choose samples from different classes as well.
- The scores for the given tasks in the notebook sum up to 70%. If they are not properly normalized in the final notebook, I'll assign a score of 20% to the 'im2col' task, and then normalize each score by a factor of 10/9 so they sum up to 100%.

Good Luck!